

Hands-on Activity 5.3	
Sounds and Effects	
Course Code: CPE 411	Program: Computer Engineering
Course Title: Interactive Extended Reality	Date Performed: 7/04/2026
Section: 22S2	Date Submitted: 7/04/2026
Name(s): Ross Andrew Bulaong	Instructor: Engr. Jimlord M. Quejado
1. Discussion Games aren't just seen—they're heard and felt. In Sound and Effects you'll add the polish that makes actions satisfying. You'll integrate background music, positional sound effects, and particle systems to emphasize key moments like jumps, collisions, and power-ups. Along the way you'll manage audio sources and mixers for volume balance, and you'll trigger VFX through simple C# events. These techniques will breathe life into your project and elevate player immersion.	
2. Materials and Equipment Unity Hub and Unity Editor Learn Unity Website (Unity Essentials Pathway - Learn Game Development for Beginners Unity Learn) Computer with Network Connection	
3. Procedure 1. Unit 3 - Introduction Q&A (0)  Mark Step As Complete Watch the introduction	

Universal 3D

The URP (Universal Render Pipeline) blank template includes the settings and assets you need to start creating with URP. Equipped with artist-...



Read more



Upgrade

Unity organization

RossUsesUnity



Project name *



Prototype3



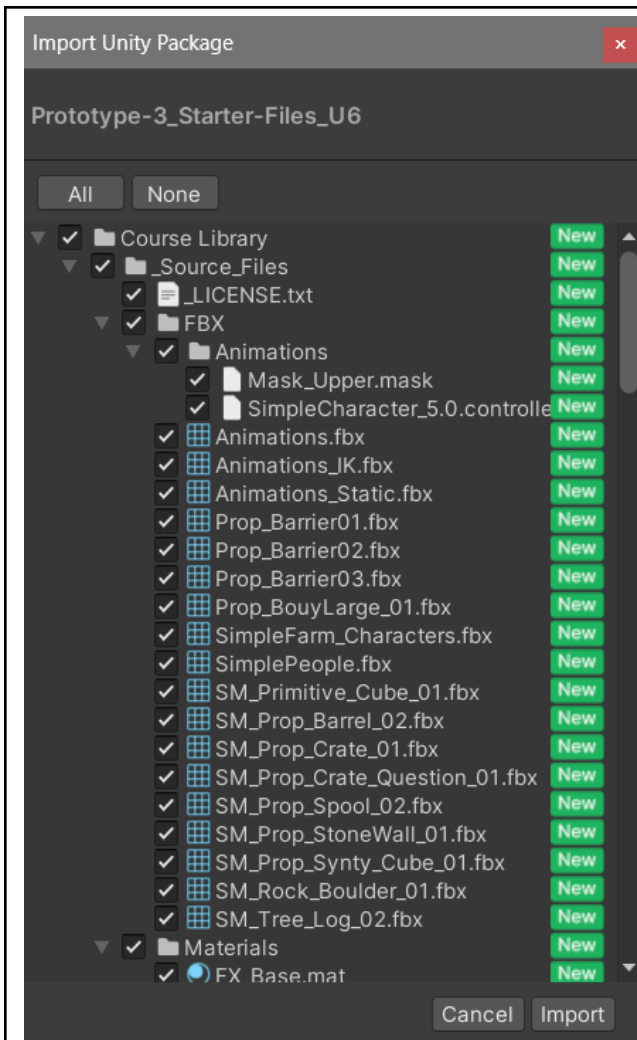
Location *

C:\Users\Andrew Bulaong\Create With ...

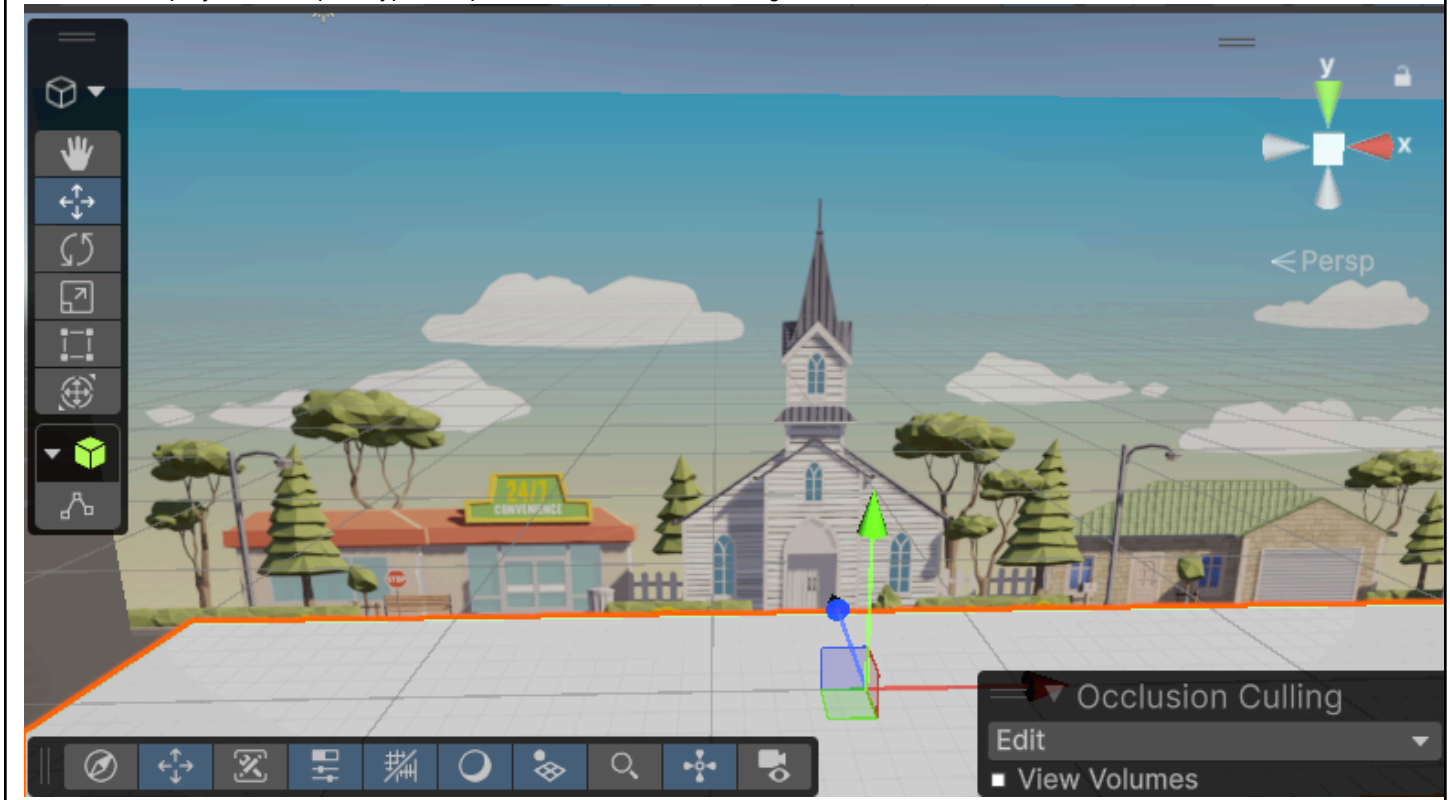


Use Unity Version Control ?

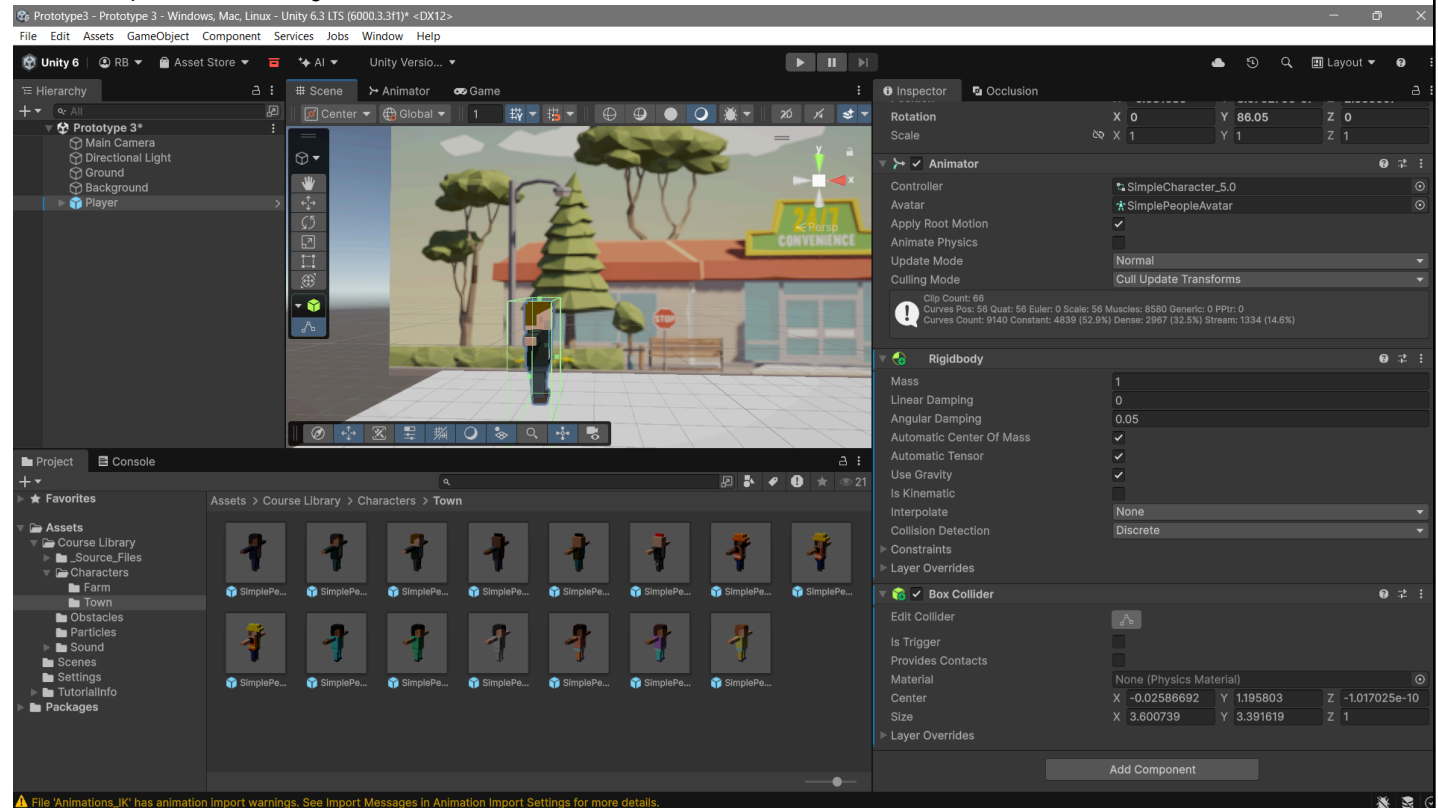
+ Create project



Create a new project called prototype3, import the starter files into the given scene.



Choose a sprite for the background



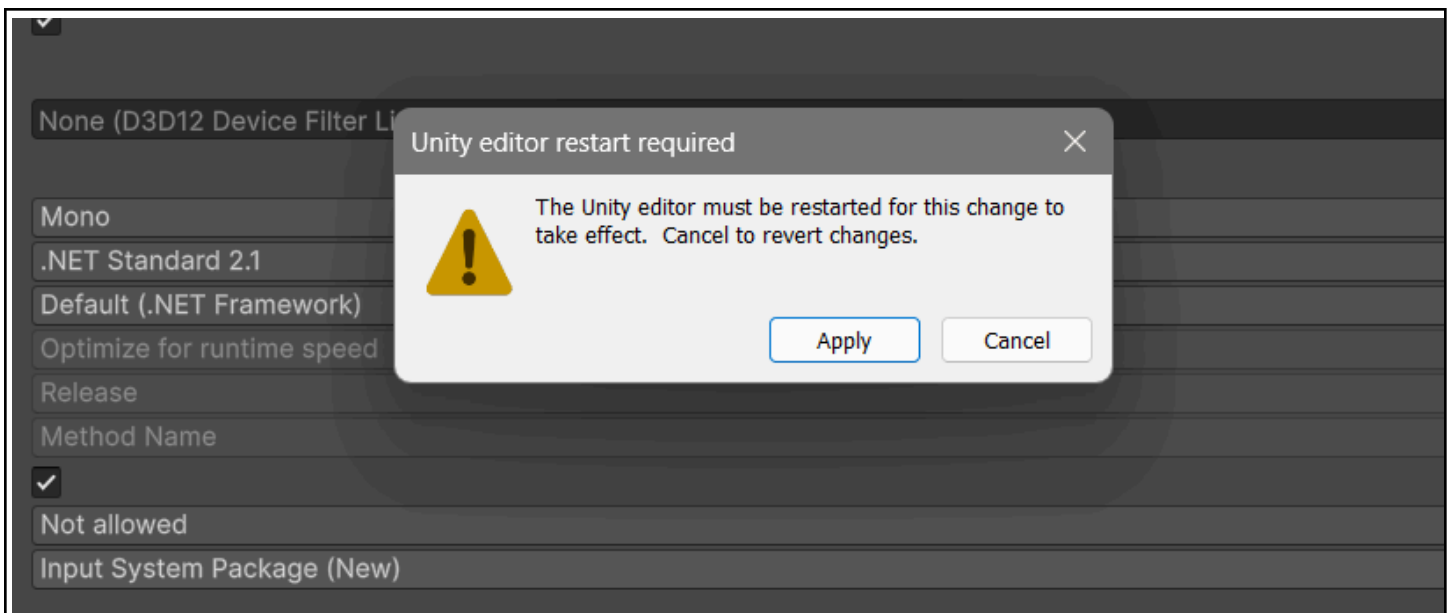
Create a player character then add a rigidbody and a box collider

```

1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      private Rigidbody playerRb;
6
7      void Start()
8      {
9          playerRb = GetComponent<Rigidbody>();
10         playerRb.AddForce(Vector3.up * 10000);
11     }
12
13
14     void Update()
15     {
16
17     }
18 }
19

```

Create a player controller script, create a private variable that gets the player (self-attached) rigid body then adds an upward force to it



Active Input Handling* Both

Change the Active Input Handling to Both

```
1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      private Rigidbody playerRb;
6
7      void Start()
8      {
9          playerRb = GetComponent<Rigidbody>();
10     }
11
12
13
14     void Update()
15     {
16         if (Input.GetKeyDown(KeyCode.Space)){
17             playerRb.AddForce(Vector3.up * 10000, ForceMode.Impulse);
18         }
19     }
20 }
21
```

Player now jumps if space is pressed

```

1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      private Rigidbody playerRb;
6      public float jumpForce = 10;
7      public float gravityModifier;
8      public bool isOnGround = true;
9
10     void Start()
11     {
12         playerRb = GetComponent<Rigidbody>();
13         Physics.gravity *= gravityModifier;
14     }
15
16
17     void Update()
18     {
19         if (Input.GetKeyDown(KeyCode.Space) && isOnGround){
20             playerRb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
21             isOnGround = false;
22         }
23     }
24

```

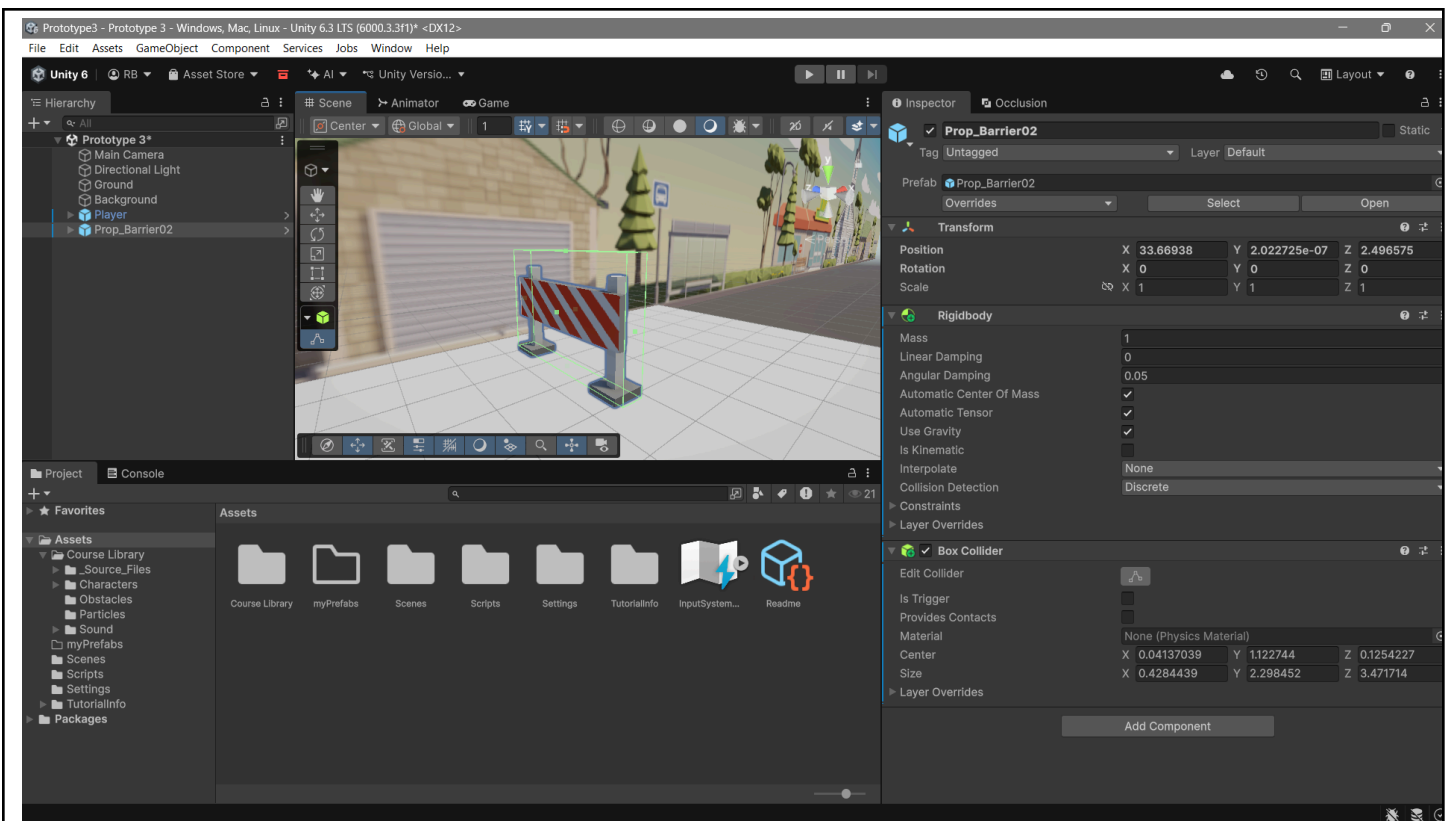
Physics of jump can now be tweaked via Inspector as well as Jump Force (how high the jump is)

```

1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      private Rigidbody playerRb;
6      public float jumpForce = 10;
7      public float gravityModifier;
8      public bool isOnGround = true;
9
10     void Start()
11     {
12         playerRb = GetComponent<Rigidbody>();
13         Physics.gravity *= gravityModifier;
14     }
15
16
17     void Update()
18     {
19         if (Input.GetKeyDown(KeyCode.Space) && isOnGround){
20             playerRb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
21             isOnGround = false;
22         }
23     }
24
25     private void OnCollisionEnter(Collision collision){
26         isOnGround = true;
27     }
28 }
29

```

Add a ground check to prevent double jumps



Add an obstacle with a rigidbody and a box collider.

```

1  using UnityEngine;
2
3  public class MoveLeft : MonoBehaviour
4  {
5      private float speed = 30f;
6      void Start()
7      {
8      }
9
10     void Update()
11     {
12         transform.Translate(Vector3.left * Time.deltaTime * speed);
13     }
14 }
15

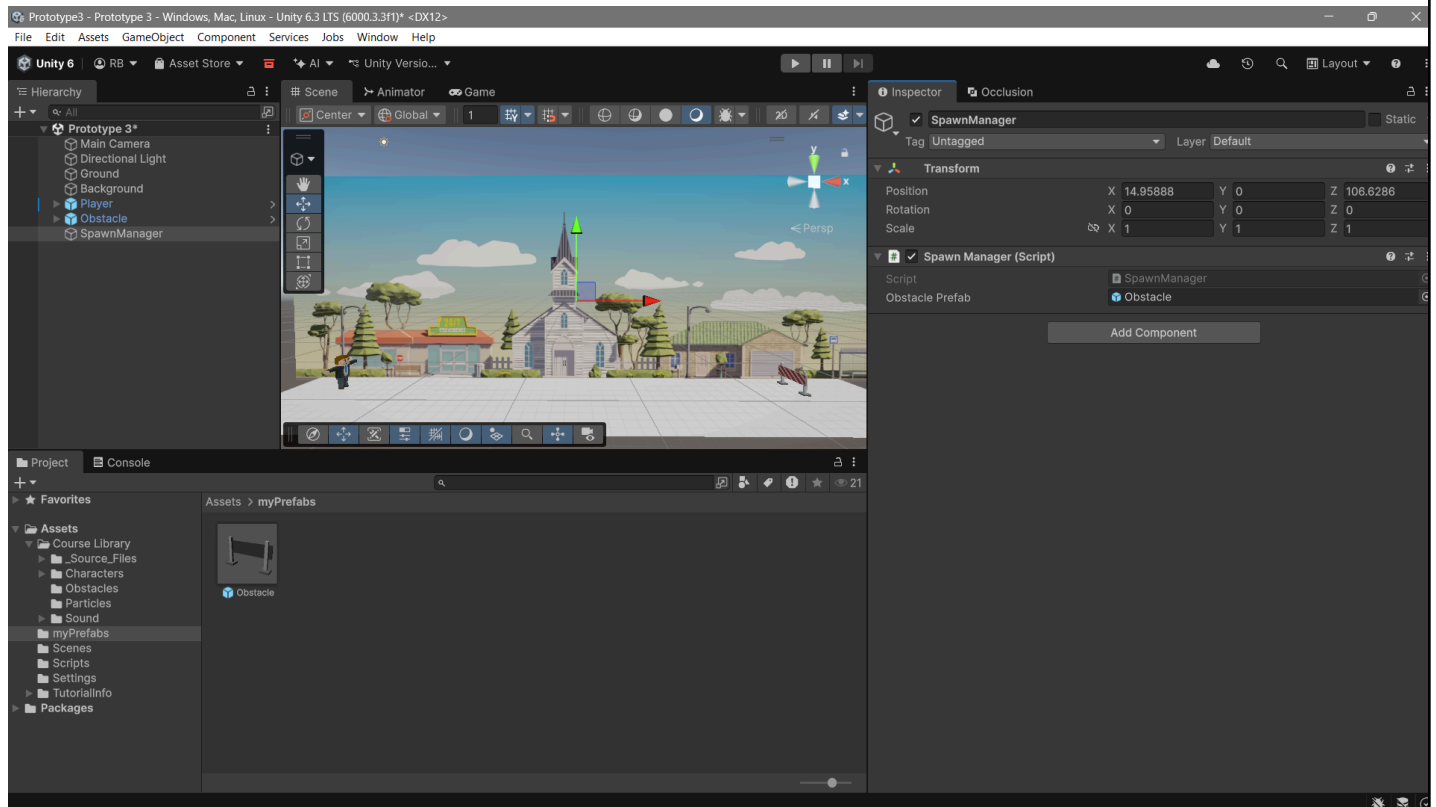
```

Create a "MoveLeft" script for obstacles so they automatically move left and attach it to the obstacle

```

1  using UnityEngine;
2
3  public class SpawnManager : MonoBehaviour
4  {
5      public GameObject obstaclePrefab;
6      private Vector3 spawnPos = new Vector3(25,0,0);
7
8      void Start()
9      {
10         Instantiate(obstaclePrefab,spawnPos, obstaclePrefab.transform.rotation);
11     }
12     void Update()
13     {
14     }
15
16 }
17

```



Create a spawn manager that creates one obstacle spawn on the spawnPos value position. In which, the object has a behavior of MoveLeft, which automatically moves it to the left.



Test the game

```
1  using UnityEngine;
2
3  public class SpawnManager : MonoBehaviour
4  {
5      public GameObject obstaclePrefab;
6      private Vector3 spawnPos = new Vector3(25,0,0);
7      private float startDelay = 2;
8      private float repeatRate = 2;
9
10     void Start()
11     {
12         InvokeRepeating("SpawnObstacle",startDelay, repeatRate);
13     }
14     private void SpawnObstacle(){
15         Instantiate(obstaclePrefab, spawnPos, obstaclePrefab.transform.rotation);
16     }
17 }
```

Modify the SpawnManager script so it automatically spawns obstacles at an interval at the start of the scene.

```

1  using UnityEngine;
2
3  public class RepeatBackground : MonoBehaviour
4  {
5      private Vector3 startPos;
6
7      void Start()
8      {
9          startPos = transform.position;
10     }
11
12     void Update()
13     {
14         if (transform.position.x < startPos.x - 50){
15             transform.position = startPos;
16         }
17     }
18 }
19

```

Create a script (and attach to background) that automatically resets the background's position back to its original state after it hits the position `startPos.x - 50` as the `MoveLeft` script moves it, well, left.

```

1  using UnityEngine;
2
3  public class RepeatBackground : MonoBehaviour
4  {
5      private Vector3 startPos;
6      private float repeatWidth;
7
8      void Start()
9      {
10         startPos = transform.position;
11         repeatWidth = GetComponent<BoxCollider>().size.x /2; }
12
13     void Update()
14     {
15         if (transform.position.x < startPos.x - repeatWidth){
16             transform.position = startPos;
17         }
18     }
19 }
20

```

Create a script that removes the awkward jerk of the update function's transition back to the original position by creating a `repeatWidth` variable that contains half the size of the background. This distance of "repeating" the background ensures its smooth return back to the original starting position.

```

1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      private Rigidbody playerRb;
6      public float jumpForce = 10;
7      public float gravityModifier;
8      private bool isOnGround = true;
9      public bool gameOver = false;
10
11
12      void Start()
13      {
14          playerRb = GetComponent<Rigidbody>();
15          Physics.gravity *= gravityModifier;
16      }
17
18
19      void Update()
20      {
21          if (Input.GetKeyDown(KeyCode.Space) && isOnGround){
22              playerRb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
23              isOnGround = false;
24          }
25      }
26
27      private void OnCollisionEnter(Collision collision){
28          if (collision.gameObject.CompareTag("Ground")){
29              isOnGround = true;
30          }
31          else if (collision.gameObject.CompareTag("Obstacle")){
32              gameOver=true;
33              Debug.Log("Game Over!");
34          }
35      }
36  }
37

```

Add more logic to the script by assigning tags to different gameObjects instead of manually checking for colliders and such. Create a gameOver variable that will likely be used to trigger other functions in other scripts.

```

1  using UnityEngine;
2
3  public class MoveLeft : MonoBehaviour
4  {
5      private float speed = 30f;
6      private PlayerController playerControllerScript;
7
8      void Start()
9      {
10         playerControllerScript = GameObject.Find("Player").GetComponent<PlayerController>();
11     }
12     void Update()
13     {
14         if (playerControllerScript.gameOver == false){
15             transform.Translate(Vector3.left * Time.deltaTime * speed);
16         }
17     }
18 }
19
20

```

Add logic to the MoveLeft script by constantly checking if the PlayerController's GameOver attribute is false. As long as its false (aka

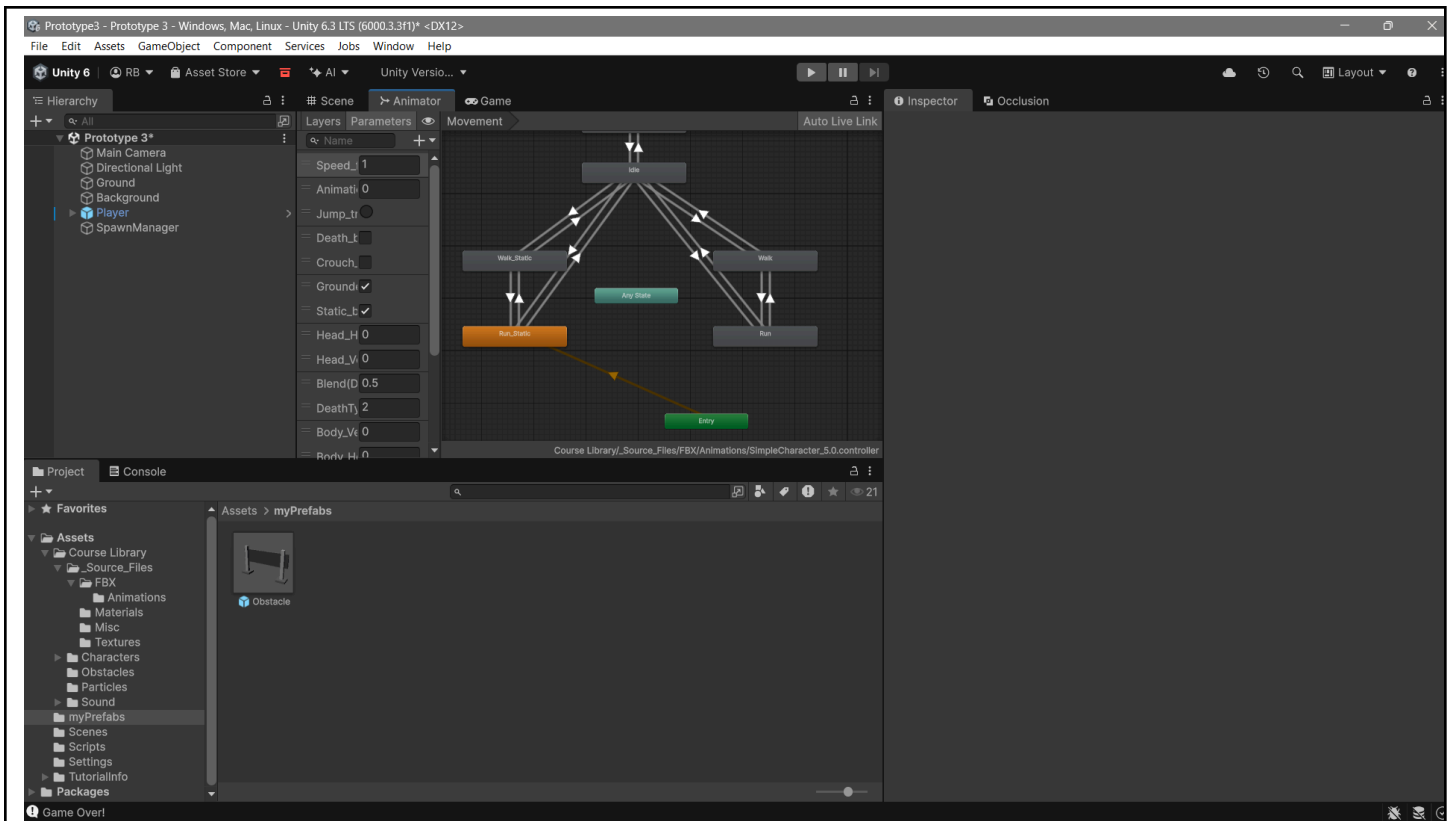
in the PlayerController, the collider of the player hasn't come into collision with the gameObject(s) with the obstacle tag, the MoveLeft will still work, i.e the background and obstacles will still move left. Otherwise, they'll stop moving (stay in place)

```
1  using UnityEngine;
2
3  public class SpawnManager : MonoBehaviour
4  {
5      public GameObject obstaclePrefab;
6      private Vector3 spawnPos = new Vector3(25,0,0);
7      private float startDelay = 2;
8      private float repeatRate = 2;
9      private PlayerController playerControllerScript;
10
11     void Start()
12     {
13         InvokeRepeating("SpawnObstacle",startDelay, repeatRate);
14         playerControllerScript = GameObject.Find("Player").GetComponent<PlayerController>();
15     }
16     private void SpawnObstacle(){
17         if (playerControllerScript.gameOver == false) {
18             Instantiate(obstaclePrefab, spawnPos, obstaclePrefab.transform.rotation);
19         }
20     }
21 }
22
23
```

Similarly, the spawn manager also checks for the PlayerController's gameOver attribute. As long as it's not gameOver == true, objects will spawn.

```
1  using UnityEngine;
2
3  public class MoveLeft : MonoBehaviour
4  {
5      private float speed = 30f;
6      private PlayerController playerControllerScript;
7      public float leftBound = -15f;
8
9      void Start()
10     {
11         playerControllerScript = GameObject.Find("Player").GetComponent<PlayerController>();
12     }
13     void Update()
14     {
15         if (playerControllerScript.gameOver == false){
16             transform.Translate(Vector3.left * Time.deltaTime * speed);
17         }
18
19         if (transform.position.x < leftBound && gameObject.CompareTag("Obstacle")) {
20             Destroy(gameObject);
21         }
22     }
23 }
24
25
```

Add logic to the MoveLeft script basically stating that if the object is an obstacle and its position is less than the leftBound, delete it. This is to avoid lagging the game computing for physics of off-screen objects.



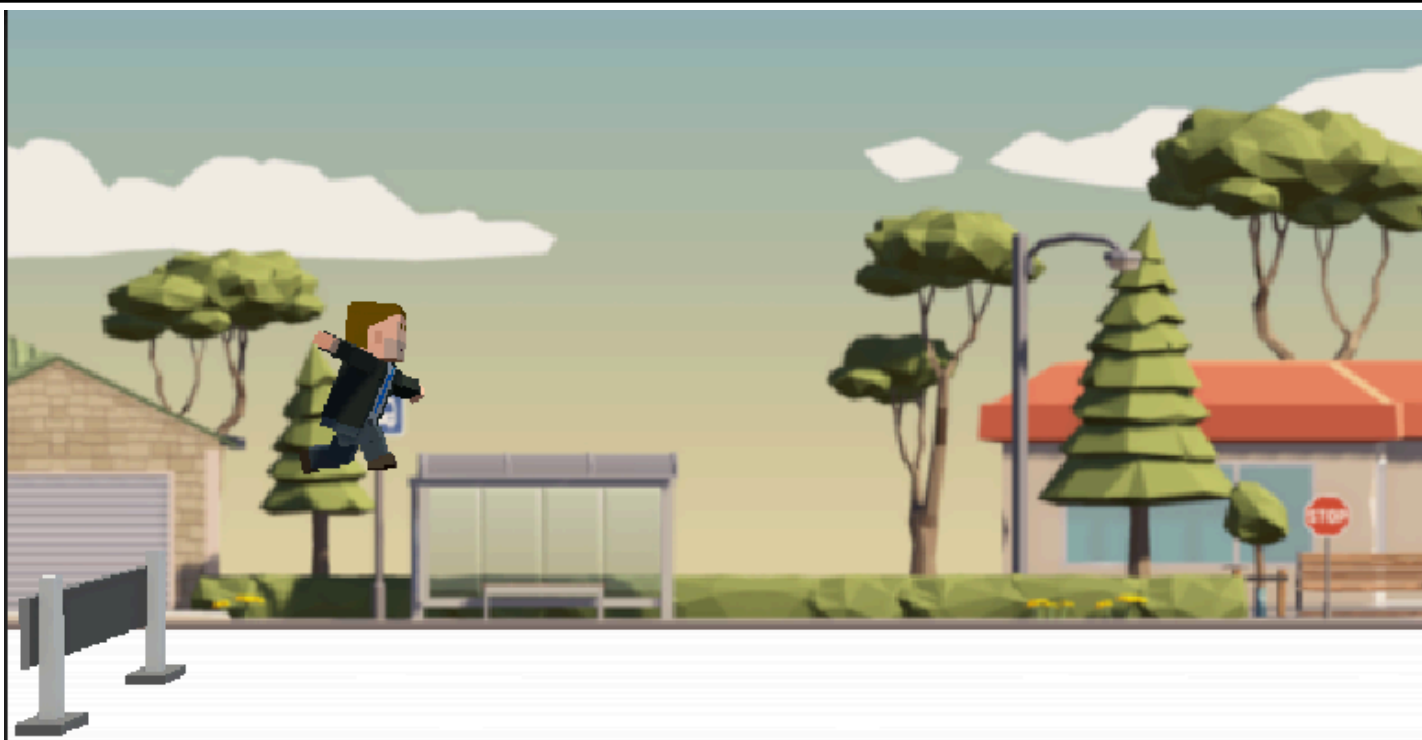
Make Run-Static as the default stage for the Player's animation (it starts at Run-Static) then i changed the speed to match the speed of the background

```

1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      private Rigidbody playerRb;
6      public float jumpForce = 10;
7      public float gravityModifier;
8      private bool isOnGround = true;
9      public bool gameOver = false;
10     private Animator playerAnim;
11
12
13     void Start()
14     {
15         playerRb = GetComponent<Rigidbody>();
16         Physics.gravity *= gravityModifier;
17         playerAnim = GetComponent<Animator>();
18     }
19
20
21     void Update()
22     {
23         if (Input.GetKeyDown(KeyCode.Space) && isOnGround){
24             playerRb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
25             isOnGround = false;
26             playerAnim.SetTrigger("Jump_trig");
27         }
28     }
29
30     private void OnCollisionEnter(Collision collision){
31         if (collision.gameObject.CompareTag("Ground")){
32             isOnGround = true;
33         }
34         else if (collision.gameObject.CompareTag("Obstacle")){
35             gameOver=true;
36             Debug.Log("Game Over!");
37         }
38     }
39 }
40

```

Adding logic to the PlayerController script by getting its assigned animator then setting the animation to the jump animation if the spacebar (jump button) is pressed.



Adjusted the player's animation, mass, rotations, angular damping, and freeze position constraints to create the perfect jump animation.



```

private void OnCollisionEnter(Collision collision){
    if (collision.gameObject.CompareTag("Ground")){
        isOnGround = true;
    }
    else if (collision.gameObject.CompareTag("Obstacle")){
        gameOver=true;
        Debug.Log("Game Over!");
        playerAnim.SetBool("Death_b", true);
        playerAnim.SetInteger("DeathType_int", 1);
    }
}

```

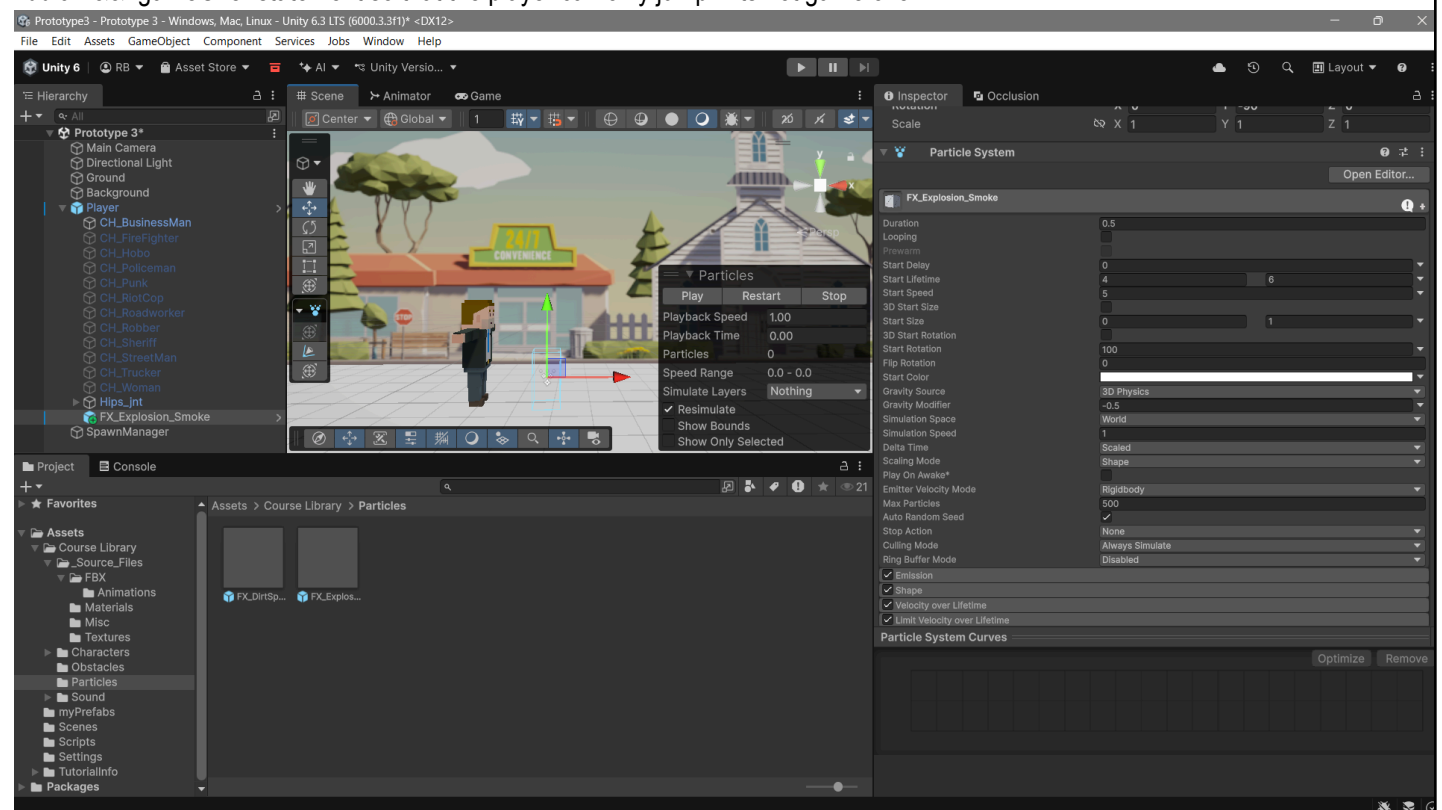
Added logic to the PlayerController so that when the player collides with an obstacle, the Death_bool sets to true and plays an animation of index 1 (Deathtype_int 1)

```

void Update()
{
    if (Input.GetKeyDown(KeyCode.Space) && isOnGround && !gameOver){
        playerRb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
        isOnGround = false;
        playerAnim.SetTrigger("Jump_trig");
    }
}

```

Add an && !gameOver statement so that the player can only jump if its not game over



Drag a particle into the player to make it a child gameObject then untick Play on Awake

```

private void OnCollisionEnter(Collision collision){
    if (collision.gameObject.CompareTag("Ground")){
        isOnGround = true;
    }
    else if (collision.gameObject.CompareTag("Obstacle")){
        explosionParticle.Play();
        gameOver=true;
        Debug.Log("Game Over!");
        playerAnim.SetBool("Death_b", true);
        playerAnim.SetInteger("DeathType_int", 1);
    }
}

```

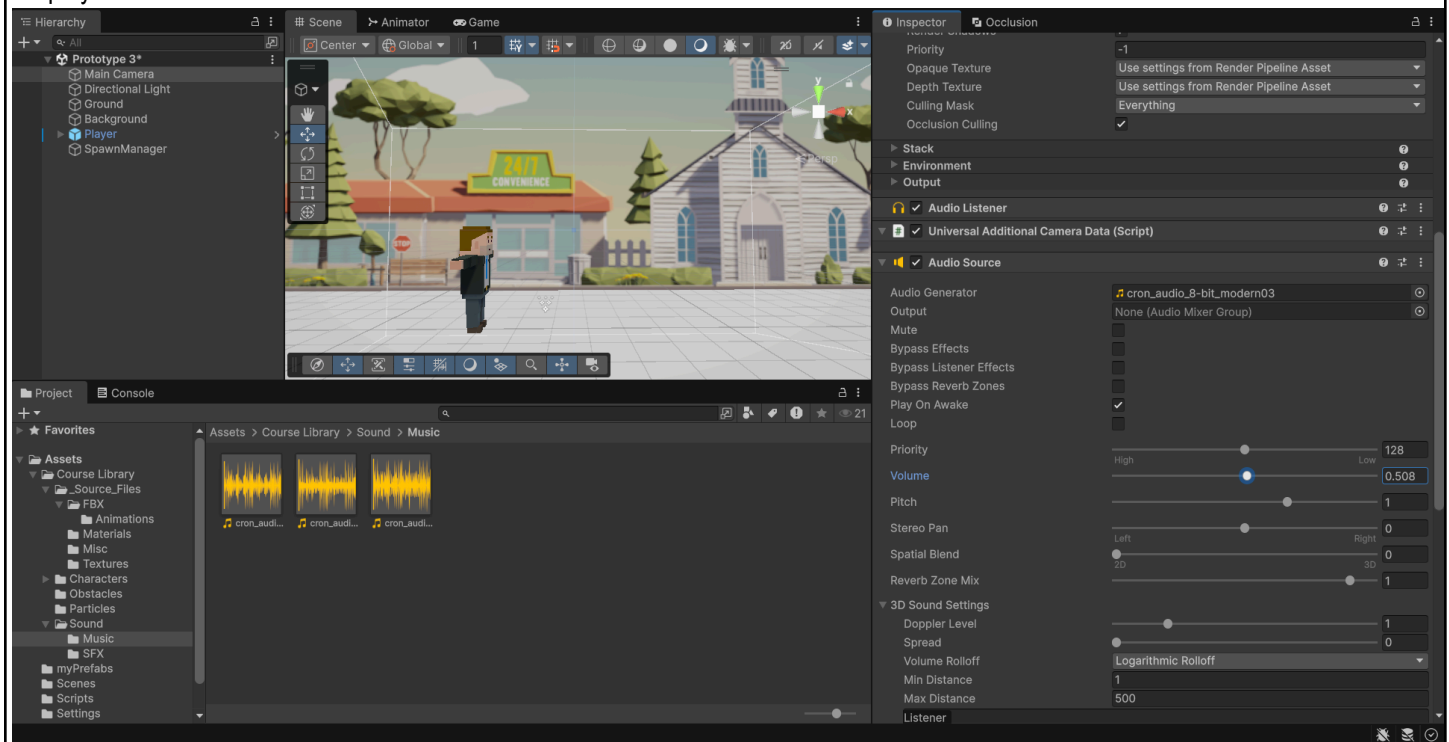
Create a public variable in the inspector where we can assign the explosionParticle that plays if the player hits an obstacle.

```

private void OnCollisionEnter(Collision collision){
    if (collision.gameObject.CompareTag("Ground")){
        isOnGround = true;
        dirtSplatterParticle.Play();
    }
    else if (collision.gameObject.CompareTag("Obstacle")){
        dirtSplatterParticle.Stop();
        explosionParticle.Play();
        gameOver=true;
        Debug.Log("Game Over!");
        playerAnim.SetBool("Death_b", true);
        playerAnim.SetInteger("DeathType_int", 1);
    }
}

```

Create another public variable in the inspector to assign the dirt particle effect, which plays if the player hits the ground, then stops if the player hits an obstacle



Add an Audio Source component to the Main Camera, assign an audio from the provided assets, then reduce the volume to better hear sound effects.

```
1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      private Rigidbody playerRb;
6      public float jumpForce = 10;
7      public float gravityModifier;
8      private bool isOnGround = true;
9      public bool gameOver = false;
10     private Animator playerAnim;
11     public ParticleSystem explosionParticle;
12     public ParticleSystem dirtSplatterParticle;
13     public AudioClip jumpSound;
14     public AudioClip crashSound;
15 }
```

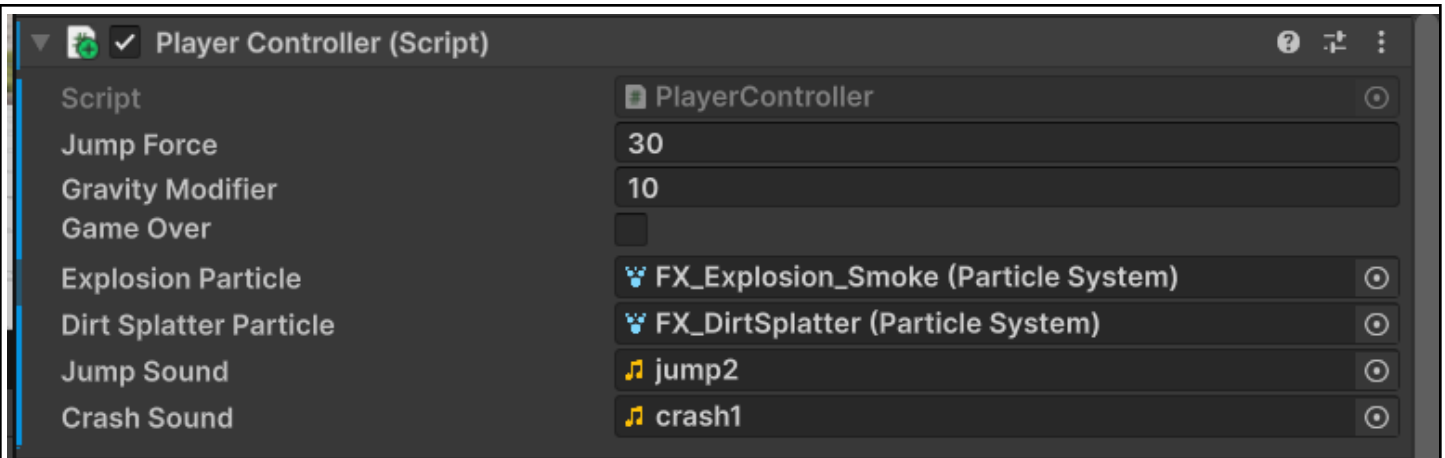
Add public variables for the jump and crash sound to be assigned via inspector.

```

1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      private Rigidbody playerRb;
6      public float jumpForce = 10;
7      public float gravityModifier;
8      private bool isOnGround = true;
9      public bool gameOver = false;
10     private Animator playerAnim;
11     public ParticleSystem explosionParticle;
12     public ParticleSystem dirtSplatterParticle;
13     public AudioClip jumpSound;
14     public AudioClip crashSound;
15     private AudioSource playerAudio;
16
17
18     void Start()
19     {
20         playerRb = GetComponent<Rigidbody>();
21         Physics.gravity *= gravityModifier;
22         playerAnim = GetComponent<Animator>();
23         playerAudio = GetComponent<AudioSource>();
24     }
25
26
27     void Update()
28     {
29         if (Input.GetKeyDown(KeyCode.Space) && isOnGround && !gameOver){
30             playerRb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
31             isOnGround = false;
32             playerAnim.SetTrigger("Jump_trig");
33             playerAudio.PlayOneShot(jumpSound,1.0f);
34         }
35     }
36
37     private void OnCollisionEnter(Collision collision){
38         if (collision.gameObject.CompareTag("Ground")){
39             isOnGround = true;
40             dirtSplatterParticle.Play();
41         }
42         else if (collision.gameObject.CompareTag("Obstacle")){
43             playerAudio.PlayOneShot(crashSound,1.0f);
44             dirtSplatterParticle.Stop();
45             explosionParticle.Play();
46             gameOver=true;
47             Debug.Log("Game Over!");
48             playerAnim.SetBool("Death_b", true);
49             playerAnim.SetInteger("DeathType_int", 1);
50         }
51     }
52 }
53

```

Add an audio source component to the player so that it can generate sounds, then, play sounds whenever the player jumps or crashes.



Assigned variables

Your results

Congratulations! You passed. If you would like to score higher, you can [retake](#) the quiz. We always save your highest score.

Passed • You got 10 of 10 correct



+30 XP

You gained 30 XP points towards Art & Animation

+30 XP

You gained 30 XP points towards Programming

You completed this quiz on April 07, 2026

Next up in Junior Programmer

Gameplay Mechanics

+700 XP • 6h 40m

[Continue to next](#)

Quiz Result

DEMONSTRATION LINK: [SoundAndEffectsDEMOBULAONG](#)

4. Conclusion

I most certainly had fun with the way this activity is structured. Everything is clear and concise, the code is clean and readable, and the results are certainly visible despite the simplicity of the code. I learned a lot especially about getting components and states to do certain things such as play death animations or sounds.

5. Assessment Rubric

Lab Activity Rubric											
Criteria		Ratings									Pts
 SO 7 PI 1	Student Outcome 7.1 Acquire and apply new knowledge from outside sources. threshold: 4.8 pts	6 pts Excellent Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently and applies knowledge learned into practice	5 pts Good Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently	4 pts Satisfactory Look beyond classroom requirements, showing interest in pursuing knowledge independently	3 pts Unsatisfactory Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	2 pts Poor Relies on classroom instruction only	1 pts Very Poor No initiative or interest in acquiring new knowledge	6 pts			
 SO 7 PI 2	Student Outcome 7.2 Learn independently threshold: 4.8 pts	6 pts Excellent Completes an assigned task independently and practices continuous improvement	5 pts Good Completes an assigned task without supervision or guidance	4 pts Satisfactory Requires minimal guidance to complete an assigned task	3 pts Unsatisfactory Requires detailed or step-by-step instructions to complete a task	2 pts Poor Shows little interest to complete a task independently	1 pts Very Poor No interest to complete a task independently	6 pts			
 SO 7 PI 3	Student Outcome 7.3 Critical thinking in the broadest context of technological change threshold: 4.8 pts	6 pts Excellent Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	5 pts Good Evaluate information from a variety of sources; formulates a clear and precise perspective.	4 pts Satisfactory Analyze information from a variety of sources; formulates a clear and precise perspective.	3 pts Unsatisfactory Apply the gathered information to formulate the problem	2 pts Poor Gather and summarized the information from a variety of sources but failed to formulate the problem	1 pts Very Poor Gather information from a variety of sources	6 pts			
 SO 7 PI 4	Student Outcome 7.4 Creativity and adaptability to new and emerging technologies threshold: 4.8 pts	6 pts Excellent Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	5 pts Good Ideas are creative and adapt the new knowledge to solve a problem or address an issue	4 pts Satisfactory Ideas are creative in solving a problem, or address an issue	3 pts Unsatisfactory Shows some creative ways to solve the problem	2 pts Poor Shows initiative and attempt to develop creative ideas to solve the problem	1 pts Very Poor Ideas are copied or restated from the sources consulted	6 pts			
Total Points: 24											

